O Aplikaciji

Limbox je web aplikacija razvijena za istoimenu firmu koja se bavi izradom sendvič panela.

Glavni cilj aplikacije je olakšati rad zaposlenicima kroz jednostavno i intuitivno korisničko sučelje, omogućiti brzu prijavu u sustav te ponuditi napredni kalkulator koji, pomoću vlastitog algoritma, minimizira gubitak materijala prilikom izračuna i naručivanja panela.

Kalkulator pruža optimizirano rješenje koje pokazuje koliko se vrsta panela može proizvesti s obzirom na količinu materijala u skladištu i veličinu narudžbe.

Slična rješenja

OptiCutter

optiCutter je online softver za optimizaciju rezanja materijala — bilo da se radi o pločama (sheet/panel), rolama ili linearnoj duljini materijala (drvo, metal, cijevi…). Radi u oblaku, nije potrebno instalirati tradicionalnu aplikaciju, može se koristiti na različitim uređajima i OS-ima.

Ukupna korist ovakvog rješenja

Velika prednost ovog projekta je svakako njegova korisnost.

Mnoge slične firme nemaju idealno rješenje za ovakvu web aplikaciju jer općenito ljudi te industrije nisu toliko upoznati s IT sektorom koji bi im značajno mogao olakšati, a i poboljšati poslovanje. Firme većinom koriste gotove softwaree za računanje koji nisu namijenjeni za računanje panela te tako dolazi do konfuzije kod radnika zbog neintuitivnosti softwarea, a osim toga dolazi do bacanja viška materijala jer takvi kalkulatori nisu optimizirani za rad s takvim varijablama.

Ludus kao grupa već stupa u kontakt sa mnogim zainteresiranim firmama na području Hrvatske i regije kako bi napravili korak naprijed za naše lokalne biznise.

Mogućnost prilagodbe

Kao što je rečeno, stupanjem u kontakt s ostalim firmama Ludus je spreman prilagoditi rješenje zaiteresiranim korisnicima prema njihovim željama.

Ludus je spreman prilagoditi kalkulator ovisno o potrebama i zanimanjem same kompanije kako bi se održao minimizirani gubitak materijala. Također spremni smo promjeniti i izgled stranice kako bi bila presonalizirana. Iako smatramo da je UI jednostavan i pregledan, Ludus je spreman napraviti i opravdane preinake na želju klijenta.

Opseg projektnog zadatka

Projektni zadatak obuhvaća izradu web aplikacije koja radnicima omogućuje jednostavno prijavljivanje u sustav, pregled i upravljanje materijalima u skladištu te korištenje naprednog kalkulatora koji pomoću optimizacijskog algoritma minimizira gubitak materijala prilikom izrade i naručivanja panela.

Uz to, aplikacija pruža administratorski pristup za nadzor nad narudžbama, zalihama, zaposlenicima i rezultatima kalkulacija te nudi jednostavno i intuitivno korisničko sučelje prilagođeno korisnicima bez tehničkog predznanja.

Cilj projektnog zadatka je razviti potpuno funkcionalan sustav koji olakšava svakodnevni rad zaposlenika, povećava učinkovitost poslovanja i doprinosi održivosti proizvodnog procesa.

Moguće nadogradnje

Trenutno se za budućnost planira izrada iOS i Android aplikacije kako bi se olakšalo praćenje na terenu. Članovi Ludusa su trenutno također u ranoj fazi razvoja aplikacije za management radnika na poslu koja bi se također mogla onda implementirati i u ovakvu mobilnu i web aplikaciju.

Funkcionalni zahtjevi

ID Zahtjeva	Opis	Prioritet	Izvor	Kriterij prihvaćanja
F-001	Sustav omogućuje unos dimenzija dijelova koje treba izrezati.	Visok	Zahtjev dionika	Dijelovi se uspješno dodaju i prikazuju u tablici “Dijelovi za rezanje”.
F-002	Sustav automatski izračunava optimalan plan rezanja uz minimalan otpad.	Visok	Zahtjev dionika	Prikazani plan sadrži raspored rezanja i postotak iskorištenosti materijala.

ID Zahtjeva	Opis	Prioritet	Izvor	Kriterij prihvaćanja
F-003	Sustav prikazuje vizualni prikaz plana rezanja (1D prikaz ploče ili duljine).	Visok	Zahtjev dionika	Generirana grafika točno prikazuje raspored dijelova i rezove.
F-004	Korisnik može uređivati i brisati prethodno unesene materijale i dijelove.	Srednji	Zahtjev dionika	Promjene se odmah odražavaju u tablicama i spremanju projekta.
F-005	Sustav pruža tablicu sa prikazom trenutnog stanja materijala.	Visok	Zahtjev dionika	Lakše praćenje stanja u skladištu.
F-006	Sustav pruža obaveznu prijavu u sustav.	Visok	Zahtjev dionika	Prijava putem maila kompanije i dodjeljene šifre.
F-007	Sustav pruža mogućnost promjene lozinke.	Visok	Zahtjev dionika	Nakon prijave u sustav, dionik može promjeniti šifru putem svog profila na stranici.
F-008	Sustav pruža administratoru mogućnost dodavanja i uklanjanja korisnika te promjenu uloge.	Visok	Zahtjev dionika	Na profilu administratora je ovo omogućeno kako bi se zaobišla zloupotreba korisničkog računa.

ID Zahtjeva	Opis	Prioritet	Izvor	Kriterij prihvaćanja
F-009	Sustav pruža pregled narudžbi od različitih naručitelja.	Visok	Zahtjev dionika	U navbaru se može odabrati opcija “Narudžbe”, pritiskom na nju otvara se page na kojem se nalaze narudžbe različitih naručitelja.
F-010	Sustav pruža sustav organizacije narudžbi.	Visok	Zahtjev dionika	Na stranici “Narudžbe” postoji sustav za filtriranje, pretragu i sortiranje narudžbi.

Ostali zahtjevi

Nefunkcionalni zahtjevi

ID Zahtjeva	Opis	Prioritet
NF-001	Sustav mora biti dostupan 24/7 s vremenom neaktivnosti manjim od 1%.	Visok
NF-002	Aplikacija mora podržavati rad na svim modernim web preglednicima.	Visok
NF-003	Sustav mora štititi korisničke podatke i omogućiti prijavu s lozinkom.	Visok

ID Zahtjeva	Opis	Prioritet
NF-004	Sučelje mora biti intuitivno i jednostavno za korištenje.	Srednji
NF-005	Sustav mora podržavati rad više korisnika istovremeno.	Visok

Zahtjevi održavanja

ID Zahtjeva	Opis	Prioritet
M-001	Sustav mora imati jasno dokumentiran kod i komentare u skladu s programerskim standardima.	Visok
M-001.1	Sustav mora imati jasnu dokmentaciju definiranu na Wikiju te jasno izražen kod.	Visok
M-002	Svi moduli sustava moraju biti razvijeni tako da se mogu testirati pojedinačno (modularnost).	Visok
M-003	Ažuriranja sustava moraju se moći provesti bez prekida rada duljeg od 20 minuta.	Visok
M-004	Struktura baze podataka mora biti održiva kroz više verzija sustava.	Srednji
M-005	Kod mora biti usklađen s odabranim frameworkom i konvencijama organizacije.	Nizak

Dionici

- Zaposlenici

- Administratori sustava
- Razvojni tim

Aktori i njihovi funkcionalni zahtjevi:

A-1 Zaposlenik (inicijator) može: * Unijeti dimenzije i količine dostupnih materijala F-001 * Unijeti dimenzije dijelova koje treba izrezati F-002 * Pokrenuti izračun optimalnog plana rezanja F-003 * Definirati širinu reznog lista i orijentaciju materijala F-004 * Pregledati vizualni prikaz plana rezanja F-005 * Primiti obavijest ako nema dovoljno materijala F-006 * Urediti i obrisati prethodno unesene materijale i dijelove F-007 * Pregledati tablicu sa stanjem materijala F-008 * Prijaviti se u sustav F-009 * Promijeniti lozinku F-010 * Pregledati narudžbe naručitelja F-012 * Organizirati/pretražiti narudžne F-013

A-3 Administrator sustava (inicijator) može: * Dodavati i uklanjati korisnike F-011 * Upravljati računima korisnika i nadzirati pristup F-009, F-010

A-5 Razvojni tim (sudionik) može: * Održavati i nadograđivati kod sustava M-001, M-001.1 * Testirati pojedinačne module sustava M-002 * Provoditi nadogradnje bez prekida rada duljeg od 10 minuta M-003 * Dokumentirati strukturu baze podataka M-004 * Uskladiti kod s frameworkom i konvencijama organizacije M-005

A-7 Baza podataka (sudionik) * Sudjeluje u svakom obrascu upotrebe spremanjem i dohvatom relevantnih podataka iz sustava.

Obrasci uporabe

1. Visokorazinski dijagram

2. Poddijagrami

Opis obrazaca uporabe

UC 1 – Pregled skladišta

Glavni sudionik: Zaposlenik
Cilj: Omogućiti pregled stanja skladišta, uključujući količine panela, RAL boje i druge materijale
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti prijavljen u sustav

Opis osnovnog tijeka: 1. Zaposlenik odabire opciju “Skladište” 2. Sustav dohvaća i prikazuje trenutno stanje svih materijala (paneli, RAL boje, količine) 3. Zaposlenik može pregledavati podatke 4. Sustav omogućuje ažuriranje prikaza ako dođe do promjena

Opis mogućih odstupanja: a) Podaci nisu dostupni → Error Boundary "Greška u tablici skladišta"

UC 2 – Pregled narudžbi

Glavni sudionik: Zaposlenik
Cilj: Omogućiti pregled svih narudžbi u sustavu
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti prijavljen

Opis osnovnog tijeka: 1. Zaposlenik odabire "Narudžbe" 2. Sustav prikazuje listu svih narudžbi s osnovnim podacima 3. Zaposlenik može pregledavati detalje pojedinih narudžbi

Opis mogućih odstupanja: b) Greška pri učitavanju → prikazuje se Error Boundary "Greška pri učitavanju narudžbi" c) prije izbrisana narudžba -> OrderNotFound "Narudžba sa zadanim ID-om ne postoji"

UC 2.1 – Pretraživanje narudžbi

Glavni sudionik: Zaposlenik
Cilj: Pronaći specifičnu narudžbu prema kriterijima
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Pregledu narudžbi"

Opis osnovnog tijeka: 1. Zaposlenik unosi kriterij pretrage (ime narudžbe tj. klijenta) 2. Sustav vrši pretragu i prikazuje rezultate 3. Zaposlenik odabire željenu narudžbu

---

UC 2.2 – Filtriranje narudžbi

Glavni sudionik: Zaposlenik
Cilj: Filtrirati narudžbe prema statusu
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti na pageu "Narudžbe"

Opis osnovnog tijeka: 1. Zaposlenik odabire filtere (status) 2. Sustav prikazuje filtriranu listu narudžbi 3. Zaposlenik može pregledavati filtrirane narudžbe

---

UC 2.3 – Dodavanje narudžbi

Glavni sudionik: Zaposlenik
Cilj: Dodati novu narudžbu u sustav
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Pregledu narudžbi"

Opis osnovnog tijeka: 1. Zaposlenik odabire "Dodaj novu narudžbu" 2. Sustav otvara formu za unos podataka o narudžbi 3. Zaposlenik unosi sve potrebne podatke i sprema narudžbu 4. Sustav dodaje narudžbu u bazu 5. Prikazuje se potvrda o uspješnom dodavanju —

UC 2.4 – Sortiranje narudžbi

Glavni sudionik: Zaposlenik
Cilj: Sortirati listu narudžbi prema odabranom kriteriju
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti na pageu "Narudžbe"

Opis osnovnog tijeka: 1. Zaposlenik odabire kriterij sortiranja (datum, status, klijent) 2. Sustav sortira i ponovno prikazuje listu narudžbi 3. Zaposlenik može pregledavati sortiranu listu

---

UC 2.5 – Prikaz statusa narudžbe

Glavni sudionik: Zaposlenik
Cilj: Prikazati trenutni status odabrane narudžbe
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti na pageu "Narudžbe"

Opis osnovnog tijeka: 1. Zaposlenik odabire narudžbu 2. Sustav prikazuje detalje narudžbe, uključujući trenutni status 3. Status se može ažurirati preko UC6.2

UC 3 – Računanje kalkulatorom

Glavni sudionik: Zaposlenik
Cilj: Izračunati potrebne materijale i troškove na temelju unesenih parametara
Sudionici: Baza podataka, kalkulator
Preduvjet: Zaposlenik mora biti prijavljen

Opis osnovnog tijeka: 1. Zaposlenik otvara kalkulator unutar aplikacije 2. Unosi dimenzije, količine i druge parametre 3. Sustav izračunava potrebne materijale i cijenu 4. Rezultati se prikazuju korisniku

Opis mogućih odstupanja: a) Neispravni ulazni podaci → prikazuje se Warning "Potrebno je ispuniti sva polja",
b) Kalkulator nije dostupan → prikazuje se obavijest

UC 4 – Usporedi sa Ludus kalkulatorom

Glavni sudionik: Zaposlenik
Cilj: Usporediti rezultate Opticutter kalkulatora s Ludus kalkulatorom
Sudionici: Opticutter kalkulator (vanjski sustav)
Preduvjet: Zaposlenik mora imati izračun u UC3

Opis osnovnog tijeka: 1. Zaposlenik odabire opciju "Usporedi" 2. Sustav šalje podatke Opticutter kalkulatoru 3. Opticutter kalkulator vraća rezultate 4. Sustav prikazuje usporedbu rezultata

Opis mogućih odstupanja: b) Podaci se ne podudaraju → prikazuje se upozorenje

---

UC 4.1 – Dohvat Opticutter rješenja

Glavni sudionik: Zaposlenik
Cilj: Dohvatiti optimizirano rješenje rezanja panela putem Opticutter alata
Sudionici: Opticutter sustav
Preduvjet: Zaposlenik mora imati unesene dimenzije u kalkulator (UC3)

Opis osnovnog tijeka: 1. Zaposlenik odabire "Usporedi" 2. Prikazuje se rezultat korisniku

---

UC 5 – Upravljaj računima

Glavni sudionik: Admin
Cilj: Upravljati korisničkim računima u sustavu
Sudionici: Baza podataka
Preduvjet: Admin mora biti prijavljen

Opis osnovnog tijeka: 1. Admin odabire "Upravljanje računima" 2. Sustav prikazuje listu svih korisnika 3. Admin može dodavati, mijenjati ili brisati račune

Opis mogućih odstupanja: a) Nema korisnika → prikazuje se prazna lista

---

UC 5.1 – Dodavanje i micanje zaposlenika

Glavni sudionik: Admin
Cilj: Dodati novog ili ukloniti postojećeg zaposlenika iz sustava
Sudionici: Baza podataka
Preduvjet: Admin mora biti na pageu svog profila

Opis osnovnog tijeka: 1. Admin odabire "Dodaj zaposlenika" ili "Ukloni zaposlenika" 2. Unose se potrebni podaci (za dodavanje) ili potvrđuje brisanje 3. Sustav ažurira bazu podataka 4. Prikazuje se potvrda o izvršenoj radnji

Opis mogućih odstupanja: a) Dupli korisnički podaci → sustav prikazuje upozorenje
b) Pokušaj brisanja nepostojećeg zaposlenika → prikazuje se greška "Greška u profilu"

UC 6 – Uređivanje narudžbe

Glavni sudionik: Zaposlenik
Cilj: Omogućiti uređivanje postojeće narudžbe
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Pregledu narudžbi" i odabrati narudžbu

Opis osnovnog tijeka: 1. Zaposlenik odabire opciju "Uredi narudžbu" 2. Sustav otvara formu s trenutnim podacima narudžbe 3. Zaposlenik mijenja potrebne podatke 4. Potvrđuje promjene 5. Sustav ažurira narudžbu u bazi

Opis mogućih odstupanja:

InvalidInput: "Validacija nije uspjela (prazna polja, negativne vrijednosti, nevažeći format)", IncorrectRALCode: "RAL kod mora biti 4-znamenkasti broj (1000-9999)", InsufficientStock: "Nije moguće smanjiti zalihe ispod nule", PanelNotFound: "Panel sa zadanim ID-om ne postoji" Environment-VariableNotSet: "Nedostaje obavezna varijabla okruženja", OrderNotFound: "Narudžba sa zadanim ID-om ne postoji",

---

UC 6.1 – Spremanje izmjena narudžbe

Glavni sudionik: Zaposlenik
Cilj: Spremiti izmjene napravljene na narudžbi
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Uređivanju narudžbe"

Opis osnovnog tijeka: 1. Nakon izmjena, zaposlenik odabire "Spremi promjene" 2. Sustav validira unesene podatke 3. Ako su podaci ispravni, spremaju se u bazu 4. Prikazuje se potvrda o uspješnom spremanju

Opis mogućih odstupanja: a) Podaci nisu ispravni → vraća se na uređivanje
b) Greška pri spremanju → prikazuje se obavijest o grešci

UC 6.2 – Promjena stanja narudžbe

Glavni sudionik: Zaposlenik
Cilj: Promijeniti status narudžbe (npr. u obradi, završeno)
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Uređivanju narudžbe"

Opis osnovnog tijeka: 1. Zaposlenik odabire novi status iz padajućeg izbornika 2. Potvrđuje promjenu 3. Sustav ažurira status narudžbe u bazi 4. Prikazuje se potvrda o promjeni statusa

Opis mogućih odstupanja: a) Nevaljani status → sustav ne dopušta promjenu
b) Narudžba je već završena → status se ne može mijenjati

UC 6.3 – Izvoz u PDF

Glavni sudionik: Zaposlenik
Cilj: Izvesti narudžbu u PDF formatu
Sudionici: Baza podataka, PDF generator
Preduvjet: Zaposlenik mora biti u "Uređivanju narudžbe"

Opis osnovnog tijeka: 1. Zaposlenik odabire "Izvezi u PDF" 2. Sustav generira PDF dokument s podacima narudžbe 3. PDF se preuzima na uređaj zaposlenika — ### UC 7 – Upravljanje skladištem

Glavni sudionik: Administrator
Cilj: Upravljati podacima o panelima i RAL-ovima u skladištu
Sudionici: Baza podataka
Preduvjet: Administrator mora biti prijavljen u sustav i imati pristup stranici Skladište

Opis osnovnog tijeka

1. Administrator otvara stranicu Skladište
2. Sustav dohvaća podatke o panelima i RAL-ovima iz baze podataka
3. Administrator odabire jednu od dostupnih radnji:
   - dodavanje novog panela
   - dodavanje novog RAL-a
   - izmjena količine postojećih panela
4. Administrator unosi potrebne podatke i potvrđuje akciju
5. Sustav provjerava ispravnost unesenih podataka
6. Sustav ažurira podatke u bazi podataka

7. Sustav prikazuje potvrdu o uspješno izvršenoj radnji

Opis mogućih odstupanja

a) Uneseni podaci nisu ispravni ili su nepotpuni
→ Sustav prikazuje poruku o pogrešci i ne sprema promjene

b) Panel ili RAL ne postoji u bazi podataka
→ Sustav prikazuje obavijest da traženi zapis nije pronađen

c) Administrator nema potrebna prava pristupa
→ Sustav odbija zahtjev i prikazuje poruku o zabrani pristupa

---

Opis mogućih odstupanja: a) Greška pri generiranju PDF-a → prikazuje se poruka o grešci
b) Nedostaju podaci za PDF → sustav zahtijeva popunu podataka

UC 7.1 – Dodavanje novih panela

Glavni sudionik: Zaposlenik
Cilj: Dodati novi tip panela u skladište
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Pregledu skladišta"

Opis osnovnog tijeka: 1. Zaposlenik odabire opciju "Dodaj novi panel" 2. Sustav otvara formu za unos podataka o panelu (dimenzije, tip, količina) 3. Zaposlenik unosi podatke i potvrđuje unos 4. Sustav sprema novi panel u bazu podataka 5. Prikazuje se poruka o uspješnom dodavanju

Opis mogućih odstupanja: InvalidInput: "Validacija nije uspjela (prazna polja, negativne vrijednosti, nevažeći format)", RALCodeNotFound: "RAL kod ne postoji u bazi podataka",

---

UC 7.2 – Dodavanje novih RAL boja

Glavni sudionik: Zaposlenik
Cilj: Dodati novu RAL boju u ponudu
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti u "Pregledu skladišta"

Opis osnovnog tijeka: 1. Zaposlenik odabire "Dodaj novu RAL boju" 2. Sustav prikazuje formu za unos RAL koda i naziva boje 3. Zaposlenik unosi podatke i sprema ih 4. Sustav dodaje boju u bazu 5. Prikazuje se potvrda o uspješnom unosu

Opis mogućih odstupanja: RALCodeAlreadyExists: "RAL kod već postoji", IncorrectRALCode: "RAL kod mora biti 4-znamenkasti broj (1000-9999)" —

UC 7.3 – Mijenjanje količine panela na stanju

Glavni sudionik: Zaposlenik
Cilj: Ažurirati količinu postojećih panela u skladištu
Sudionici: Baza podataka
Preduvjet: Zaposlenik mora biti na pageu "Skladište"

Opis osnovnog tijeka: 1. Zaposlenik odabire panel čiju količinu želi mijenjati 2. Odabire opciju "Uredi količinu" 3. Unosi novu količinu i potvrđuje promjenu 4. Sustav ažurira podatke u bazi 5. Prikazuje se potvrda o ažuriranju

Opis mogućih odstupanja: InsufficientStock: "Nije moguće smanjiti zalihe ispod nule"

Arhitektura sustava

Opis

Sustav je razvijen prema principu slojevite arhitekture, koja se sastoji od tri glavna dijela: presentation layer (frontend), business logic layer (backend) i data access layer (baza podataka). Ovakva arhitektura omogućuje jasnu podjelu odgovornosti između slojeva, gdje svaki ima precizno definiranu ulogu i način komunikacije s ostalima. Prednost ovog pristupa leži u odvajanju briga (separation of concerns), što doprinosi lakšem održavanju, većoj skalabilnosti i fleksibilnosti sustava.

Podsustavi:

- Usluga upravljanjem skladištem: omogućuje praćenje stanja na skladištu i dodavanje/uređivanje vrsta materijala u skladištu.
- Usluga upravljanja korisnicima: upravlja kreiranjem i brisanjem korisničkih profila.
- Usluga kalkulatora: omogućuje izračun pošiljke uz minimalnu materijalnu potrošnju.
- Usluga autentifikacije i autorizacije: upravlja prijavom korisnika, autentifikacijom i autorizacijom pomoću usluge OAuth2 koja koristi JSON Web Token (JWT) za provjeru korisničkih zahtjeva.

Spremišta podataka: Koristi se relacijska baza pokrenuta kroz PostgreSQL.

Mrežni protokoli: Koriste se TCP/IP i HTTP protokol. Kada korisnik na korisničkom sučelju (UI) izvrši neku akciju, primjerice klikne na određeni element, aplikacija šalje API poziv, korišten preko RESTful ahritekture, prema backendu putem HTTP zahtjeva u formatu JSON. Backend zatim prima taj zahtjev, obrađuje ga — izvršava potrebnu poslovnu logiku, pristupa bazi podataka pomoću TCP/IP protokola ako je potrebno — te nakon obrade vraća odgovor

(response) također u JSON formatu. Nakon što frontend primi taj odgovor, ažurira korisničko sučelje i prikazuje promjene korisniku u stvarnom vremenu.

Sklopovskoprogramski zahtjevi:

Obrazloženje odabira arhitekture

Glavni razlozi za odabir slojevite arhitekture bili su **skalabilnost**, **modularnost** i **odvajanje briga (separation of concerns)**. Budući da je cilj aplikacije pružiti stabilno i dugoročno rješenje koje će se moći prilagođavati različitim korisnicima i budućim funkcionalnostima, bilo je nužno odabrati arhitekturu koja podržava takav rast i fleksibilnost.

U sloju frontend-a, razvijenom u **Reactu**, korisnicima se nudi intuitivno i responzivno sučelje koje omogućuje jednostavnu interakciju s aplikacijom. Sloj backend-a, implementiran u **Kotlinu**, odgovoran je za poslovnu logiku i komunikaciju s bazom podataka. Za data access layer korišten je **PostgreSQL**, koji osigurava pouzdano i učinkovito upravljanje podacima. Komunikacija između slojeva odvija se putem **HTTP zahtjeva** u **JSON** formatu, čime se postiže jasno definiran i standardiziran način razmjene podataka.

Prilikom oblikovanja sustava primijenjeni su ključni principi **softverskog dizajna**:

- **Visoka kohezija** – svaki sloj ima jasno definiranu ulogu i funkcionalnosti usko povezane s njegovom svrhom.
- **Niska povezanost (low coupling)** – slojevi su međusobno neovisni, što omogućuje lakše održavanje i izmjene bez utjecaja na druge dijelove sustava.
- **Fleksibilnost** – sustav je jednostavno moguće nadograditi novim funkcionalnostima ili zamijeniti tehnologiju pojedinog sloja bez većih promjena u ostatku aplikacije.
- **Sigurnost i stabilnost** – backend sloj kontrolira pristup bazi podataka i validaciju zahtjeva, čime se sprječavaju neautorizirani pristupi i osigurava pouzdanost podataka.

Organizacija sustava na visokoj razini

Klijent – poslužitelj Na klijentskoj strani nalazi se web aplikacija izrađena u **Reactu**, koja korisniku pruža intuitivno i responzivno grafičko sučelje. Kada korisnik izvrši neku akciju (npr. prijavu, unos materijala, pokretanje kalkulatora), frontend šalje **HTTP zahtjev** u formatu **JSON** prema backend poslužitelju. Poslužitelj, razvijen u **Kotlinu**, prima zahtjev, obrađuje podatke pomoću poslovne logike, po potrebi pristupa bazi podataka te vraća rezultat natrag klijentu u obliku JSON odgovora. Na taj način korisničko sučelje uvijek prikazuje ažurirane informacije.

Baza podataka Za pohranu i upravljanje podacima koristi se relacijska baza podataka PostgreSQL. Baza sadrži sve ključne entitete sustava, poput podataka o materijalima, narudžbama, korisnicima i rezultatima kalkulacija. Relacijski model baze omogućuje pouzdanost, konzistentnost i jednostavno povezivanje podataka, što je ključno za točnost algoritma i stabilnost cijelog sustava.

Datotečni sustav Aplikacija trenutno ne koristi zaseban **datotečni sustav** za pohranu podataka. Svi relevantni podaci pohranjuju se u relacijsku bazu PostgreSQL.

Grafičko sučelje Grafičko korisničko sučelje (GUI) izrađeno je kao web aplikacija pomoću Reacta. Sučelje je jednostavno, pregledno i prilagođeno korisnicima bez tehničkog predznanja. Povezano je s backendom putem RESTful API-ja, čime se omogućuje dvosmjerna komunikacija između klijenta i poslužitelja. Frontend dinamički prikazuje podatke koje dobiva iz backend-a te reagira na korisničke akcije u stvarnom vremenu, što korisnicima omogućuje brz i intuitivan rad unutar sustava.

Organizacija aplikacije

Frontend (React) upravlja korisničkim sučeljem Limbox aplikacije. Odgovoran je za prikaz elemenata, navigaciju kroz sučelje i omogućavanje korisniku da obavlja radnje poput prijave, pregleda stanja na skladištu, kalkulator. Kada korisnik napravi neku akciju, frontend šalje HTTP zahtjev backendu putem RESTful API-ja.

Backend (Kotlin) je u Limboxu organiziran u tri sloja po featureu: controller, service i repository. Kada zahtjev stigne do kontrolera, on ga tumači i prosljeđuje service sloju gdje se nalazi poslovna logika aplikacije, primjerice login korisnika. Ako je potrebno doći do podataka, service sloj komunicira s repository slojem koji radi direktno s bazom podataka ili vanjskim servisom. Nakon obrade, service vraća rezultat kontroleru koji ga šalje natrag frontendu kako bi korisnik dobio ažurirani prikaz u aplikaciji.

Limbox koristi MVC arhitekturu:

• Model predstavlja podatke i logiku. • View je React korisničko sučelje koje prikazuje stanje sustava i omogućava interakciju. • Controller spaja sve zajedno jer prima vanjske zahtjeve i povezuje ih s poslovnom logikom u Kotlinu.

Ovakav pristup omogućava jasno odvajanje uloga: View brine o prikazu, Model o podacima i strukturi, a Controller o komunikaciji i obradi akcija. Zahvaljujući tome, razvoj Limboxa je jednostavniji jer se frontend i backend mogu razvijati paralelno, kod je modularan, lak za održavanje i testiranje, a pojedini dijelovi sustava mogu se ponovno koristiti i proširivati bez potrebe za velikim prepravkama.

Dijagram visoke razine

Baza podataka

Opis baze

FLYWAY_SCHEMA_HISTORY

Atribut	Tip podatka	Opis varijable
version	VARCHAR(50)	Verzija migracije
description	VARCHAR(200)	Opis migracije
type	VARCHAR(20)	Vrsta migracije
script	VARCHAR(1000)	Naziv skripte
checksum	INTEGER	Kontrolni zbroj skripte
installed_by	VARCHAR(100)	Izveo/la instalaciju
installed_on	TIMESTAMP	Datum instalacije
execution_time	INTEGER	Vrijeme izvršavanja
success	BOOLEAN	Status migracije
installed_rank	INTEGER	Redoslijed instalacije

PANELS

Atribut	Tip podatka	Opis varijable
name	VARCHAR(200)	Naziv panela
thickness	INTEGER	Debljina panela u mm
ral_code	INTEGER	RAL kod (FK prema ral_codes)
length	NUMERIC(10,2)	Duljina panela u metrima
in_stock	INTEGER	Ukupan broj komada
damaged	INTEGER	Broj oštećenih komada
created_at	TIMESTAMP	Datum kreiranja
id	INTEGER	Primarni ključ

RAL_CODES

Atribut	Tip podatka	Opis varijable
ral_kod	INTEGER	RAL kod (primarni ključ)
hex_kod	VARCHAR(7)	Hex prikaz boje
created_at	TIMESTAMP	Datum kreiranja

ROLES

Atribut	Tip podatka	Opis varijable
role_name	VARCHAR(50)	Naziv uloge
permissions	TEXT	Popis dozvola
created_at	TIMESTAMP	Datum kreiranja
id	INTEGER	Primarni ključ

SCREWS

Atribut	Tip podatka	Opis varijable
type	VARCHAR(100)	Tip vijka
name	VARCHAR(100)	Naziv modela vijka
thread	VARCHAR(50)	Navoj u mm
length	INTEGER	Duljina vijka u mm
pieces_per_pack	INTEGER	Broj u pakiranju
abbreviation	VARCHAR(100)	Kratica modela
price_per_100	NUMERIC(10,2)	Cijena za 100 komada
created_at	TIMESTAMP	Datum kreiranja
id	INTEGER	Primarni ključ

USERS

Atribut	Tip podatka	Opis varijable
username	VARCHAR(50)	Korisničko ime
firstname	VARCHAR(50)	Ime korisnika
lastname	VARCHAR(50)	Prezime korisnika
password	VARCHAR(100)	Hashirana lozinka
created_at	TIMESTAMP	Datum kreiranja
role_id	INTEGER	Uloga korisnika (FK prema roles)
id	INTEGER	Primarni ključ

ORDERS

Atribut	Tip podatka	Opis varijable
name	VARCHAR(200)	Ime narudžbe
demand_map	TEXT	Mapa potražnje (panela koje treba izrezati)
result	TEXT	Rezultat Ludus kalkulatora
message	VARCHAR(50)	Poruka o uspješnosti kalkulacije
created_at	TIMESTAMP	Datum kreiranja
id	INTEGER	Primarni ključ

Dijagram baze

Relacijska shema baze:

Oznake baze:

ER dijagram:

Dijagram razreda

Dijagram stanja za Narudžbe

Dijagram Aktivnosti

Dijagram komponenata

Dijagram razmještaja

UML dijagram razmještaja prikazuje fizičku i virtualnu raspodjelu komponenti sustava unutar infrastrukture. Njime se opisuje način na koji su pojedine komponente aplikacije raspoređene na poslužiteljima, u okruženju oblaka te na uređajima krajnjih korisnika, kao i način njihove međusobne komunikacije. Prikazani dijagram izrađen je u implementacijskom obliku te pruža detaljan uvid u infrastrukturu sustava. On vjerno odražava stvarni raspored komponenti i njihovu povezanost u okviru implementirane web aplikacije.

Ovo poglavlje opisuje provedena ispitivanja implementiranih funkcionalnosti na razini komponenti i sustava. Fokus je na odabiru i izvedbi ispitnih slučajeva koji obuhvaćaju redovne, rubne uvjete i testiranje grešaka, kao i upotrebu odgovarajućih alata za provedbu testiranja.

Ispitivanje komponenti

Odrađeno je ispitivanje na 4 ključne komponente sustava. Radi se o različitim testovima koji ispituju najvažnije funkcionalnosti komponenti pri kontroliranim i nekontroliranim uvjetima. Testovi pokrivaju autentifikaciju, kalkulacije za optimizaciju rezanja, upravljanje zalihama te izračun iskoristivosti materijala. Radi veličine kodova testova oni nisu na ovoj stranici već se nalaze u direktoriju `src/test/kotlin/`.

Ukupno testova: 10

Rezultat izvođenja:

```
./gradlew test
BUILD SUCCESSFUL
10 tests completed, 0 failed
```

Izvještaj testiranja: AuthServiceTest

Testiranje kritične komponente za autentifikaciju i autorizaciju korisnika.

Ispitni slučaj 1: Prijava s ispravnim podacima

Ulaz: - Korisničko ime: “testuser” - Lozinka: “password123”

Očekivani rezultat: - Aplikacija validira lozinku putem BCrypt hash funkcije - Generira JWT token s korisničkim podacima - Vraća objekt `LoginResponse` s tokenom i korisnički ID: 1

Rezultat: Prolazi. Korisnik je uspješno prijavljen. JWT token je generiran i sadrži sve potrebne tvrdnje (claims) za autorizaciju. BCrypt validacija lozinke prolazi uspješno.

Ispitni slučaj 2: Prijava s neispravnom lozinkom

Ulaz: - Korisničko ime: “testuser” - Lozinka: “wrongpassword”

Očekivani rezultat: - Aplikacija provjerava lozinku koristeći BCrypt.checkpw() - Metoda vraća `false` jer lozinka ne odgovara hash-u u bazi - Sustav odbija prijavu i vraća `null`

Rezultat: Prolazi. Sustav ispravno odbija prijavu s neispravnom lozinkom. Ovo je kritična sigurnosna provjera koja sprječava neovlašteni pristup sustavu. BCrypt osigurava da se originalne lozinke nikada ne uspoređuju direktno.

Izvještaj testiranja: CalculatorRepositoryTest

Testiranje repozitorija za kalkulacije optimizacije rezanja materijala.

Ispitni slučaj 1: Izračun ukupne duljine dijelova

Ulaz: - Lista zahtjeva (`DemandMapItem`) koja sadrži: - Dio 1: duljina 100.0 mm, količina 5 komada - Dio 2: duljina 200.0 mm, količina 3 komada - Panel tip: “Panel1”, debljina: 100, RAL kod: 9016

Očekivani rezultat: - Aplikacija izračunava: $(100.0 \times 5) + (200.0 \times 3) = 1100.0$ mm - Metoda `getTotalPartsLength()` vraća točnu ukupnu duljinu - Rezultat se koristi za procjenu potrebne količine materijala

Rezultat: Prolazi. Kalkulacija je točna i ključna za određivanje količine materijala potrebnog za proizvodnju. Ova metoda je temelj za sve daljnje optimizacije i planiranje nabave.

Ispitni slučaj 2: Izračun ukupne duljine iskorištenog materijala

Ulaz: - Lista rezultata kalkulacije (`CalcResult`) s rezovima: - Uzorak s 2 repa (pattern: [2, 1]) - Stock duljina: 500.0 mm - Količina uzoraka: 2

Očekivani rezultat: - Aplikacija izračunava: 500.0 × 2 = 1000.0 mm - Metoda `getUsedStocksLength()` vraća ukupnu iskorištenu duljinu - Podatak omogućuje izračun efikasnosti (yield)

Rezultat: Prolazi. Ova kalkulacija omogućava praćenje koliko materijala je stvarno iskorišteno u procesu rezanja, što je ključno za analizu otpada i optimizaciju troškova.

Izvještaj testiranja: OrderResultTest

Testiranje izračuna iskoristivosti materijala (yield) - ključne poslovne metrike.

Ispitni slučaj 1: Izračun standardne iskoristivosti

Ulaz: - Ukupna duljina potrebnih dijelova: 800.0 mm - Ukupna duljina iskorištenog materijala: 1000.0 mm

Očekivani rezultat: - Formula: (800.0 / 1000.0) × 100 = 80.0% - Svojstvo `yield` objekta `OrderResult` vraća 80.0 - Iskoristivost od 80% znači 20% otpada

Rezultat: Prolazi. Iskoristivost od 80% je realistična vrijednost u industriji rezanja. Sustav točno izračunava koliki je postotak materijala stvarno iskorišten, što omogućuje vlasniku tvrtke procjenu profitabilnosti i potrebu za optimizacijom.

Ispitni slučaj 2: Izračun iskoristivosti s nula iskorištenog materijala (rubni uvjet)

Ulaz: - Ukupna duljina potrebnih dijelova: 800.0 mm - Ukupna duljina iskorištenog materijala: 0.0 mm

Očekivani rezultat: - Dijeljenje s nulom: 800.0 / 0.0 - Kotlin/Java vraća `Double.POSITIVE_INFINITY` - Sustav ne pada, već rukuje iznimnim slučajem

Rezultat: Prolazi. Sustav ispravno rukuje matematičkim rubnim uvjetom dijeljenja s nulom. Umjesto pada aplikacije, vraća beskonačnu vrijednost (`Infinity`) koja se može dodatno obraditi u business logici ili prikazati korisniku kao greška.

Izvještaj testiranja: StorageServiceTest

Testiranje servisa za upravljanje skladištem - kritična komponenta za održavanje integriteta zaliha.

Ispitni slučaj 1: Dodavanje panela s neispravnom debljinom

Ulaz: - Naziv panela: "TestPanel" - Debljina: 0 mm (nevažeća vrijednost) - RAL kod: 9016 - Duljina: 100.0 mm - Količina na zalihi: 10

Očekivani rezultat: - Aplikacija provjerava poslovni uvjet: `thickness > 0` - Validacija ne prolazi - Vraća `ServiceResult.Error` s porukom: "Thickness must be greater than 0" - Panel se **ne** dodaje u bazu podataka

Rezultat: Prolazi. Validacija unosa je kritična za održavanje kvalitete podataka. Panel s debljinom 0 mm ne može postojati u stvarnosti, pa sustav ispravno odbija takav unos i sprječava unos nelogičnih podataka u skladište.

Ispitni slučaj 2: Dodavanje panela s negativnom količinom zaliha

Ulaz: - Naziv panela: "TestPanel" - Debljina: 100 mm - RAL kod: 9016 - Duljina: 100.0 mm - Količina na zalihi: -5 (negativna vrijednost)

Očekivani rezultat: - Aplikacija provjerava: `inStock >= 0` - Validacija ne prolazi - Vraća `ServiceResult.Error` s porukom: "Stock quantity cannot be negative" - Sprječava logički nemoguće stanje u sustavu

Rezultat: Prolazi. Negativne zalihe su logički nemoguće. Ovaj test osigurava da sustav održava integritet podataka i sprječava unos vrijednosti koje bi mogle dovesti do grešaka u daljnjoj poslovnoj logici (npr. prodaja nepostojećeg proizvoda).

Ispitni slučaj 3: Prilagodba zaliha s nedovoljnom količinom (kritična poslovna logika)

Ulaz: - ID panela: 1 - Trenutna količina na zalihi: 5 komada - Pokušaj oduzimanja: -10 komada

Očekivani rezultat: - Aplikacija dohvaća trenutno stanje: 5 komada - Izračunava novo stanje: 5 + (-10) = -5 - Provjerava uvjet: `newStock < 0` → true - Odbija transakciju s greškom: "Insufficient stock" - **Zalihe ostaju nepromijenjene na 5 komada**

Rezultat: Prolazi. Ovo je najkritičnija zaštita u sustavu upravljanja zalihama. Sprječava situacije gdje bi se pokušala prodati/koristiti roba koja ne postoji na zalihi. Bez ovog testa, sustav bi mogao doći u nekonzistentno stanje gdje su zalihe negativne, što bi uzrokovalo probleme u cijelom poslovnom procesu.

Ispitni slučaj 4: Uspješna prilagodba zaliha

Ulaz: - ID panela: 1 - Trenutna količina na zalihi: 10 komada - Dodavanje: +5 komada

Očekivani rezultat: - Aplikacija dohvaća trenutno stanje: 10 komada - Izračunava novo stanje: 10 + 5 = 15 komada - Provjerava uvjet: `newStock >= 0` → true - Ažurira zalihe u bazi podataka - Vraća `ServiceResult.Success(true)`

Rezultat: Prolazi. Ovo je normalan tijek rada - uspješno dodavanje novih panela u skladište nakon nabave ili povrata. Sustav ispravno ažurira količine i omogućava daljnju prodaju/korištenje materijala.

Sažetak testiranja komponenti

Komponenta	Broj testova	Status	Ključne funkcionalnosti
AuthServiceTest	2	Prolaze	Autentifikacija, BCrypt validacija, JWT generiranje
CalculatorRepositoryTest	2	Prolaze	Kalkulacije duljina, priprema za optimizaciju
OrderResultTest	2	Prolaze	Yield izračun, rukovanje dijeljenjem s nulom
StorageServiceTest	4	Prolaze	Validacija unosa, zaštita integriteta zaliha
UKUPNO	**10**	**Svi prolaze**	**Pokrivene kritične poslovne funkcionalnosti**

Ispitivanje sustava

Cilj ispitivanja sustava je testiranje ponašanja cijelog sustava u uvjetima stvarnog korištenja, uz posebnu pažnju na međusobnu povezanost svih komponenti. Ispitivanje obuhvaća sve aspekte sustava i njegovu interakciju s korisnicima kroz REST API endpointe.

Ispit 1: Kreiranje narudžbe (redovni slučaj)

Ulazi:

Podaci za kreiranje narudžbe: - Naziv: “Narudžba panela za projekt Zagreb” - Mapa zahtjeva (`demandMap`): - Panel tip: “ALU panel”, debljina 100mm, RAL 9016 - Dijelovi: - Dio 1: duljina 1500mm, količina 10, oznaka “Panel A” - Dio 2: duljina 2000mm, količina 5, oznaka “Panel B”

Koraci ispitivanja: 1. Pokrenuti backend aplikaciju 2. Autentificirati se kao korisnik putem `/auth/login` endpointa 3. Poslati POST zahtjev na `/api/calculator/orders` s podacima narudžbe 4. Provjeriti status odgovora 5. Provjeriti kreirani objekt narudžbe

Očekivani izlaz: - API vraća status `201 Created` - Vraća objekt narudžbe s: - Generiranim ID-jem - Statusom “Pending” - Izračunatim rezultatom optimizacije - Podacima o potrebnim zalihama - Izračunatim yield-om

Dobiveni izlaz: Uspjeh. Narudžba je kreirana s ID-jem 1, status “Pending”. Sustav je automatski izračunao optimalan raspored rezanja i vratio potrebne količine materijala.

Ispit 2: Kreiranje narudžbe s nevažećim podacima (rubni uvjet)

Ulazi:

Podaci za kreiranje narudžbe: - Naziv: “” (prazan string) - Mapa zahtjeva: prazna lista

Koraci ispitivanja: 1. Autentificirati se kao korisnik 2. Poslati POST zahtjev na `/api/calculator/orders` s nevažećim podacima 3. Provjeriti status odgovora i poruku greške

Očekivani izlaz: - API vraća status `400 Bad Request` - Vraća poruku greške: “Name cannot be blank” ili “Demand list is empty”

Dobiveni izlaz: Uspjeh. Sustav je ispravno odbio kreiranje narudžbe i vratio status 400 s jasnom porukom greške. Validacija na razini servisa radi ispravno.

Ispit 3: Prilagodba zaliha panela (redovni slučaj)

Ulazi:

Podaci: - Panel ID: 1 - Promjena količine: +20 komada

Koraci ispitivanja: 1. Autentificirati se kao administrator 2. Dohvatiti trenutno stanje panela GET `/api/storage/panels/1` 3. Poslati PATCH zahtjev na `/api/storage/panels/1/adjust-stock` s količinom +20 4. Provjeriti ažurirano stanje

Očekivani izlaz: - API vraća status `200 OK` - Zalihe panela se povećavaju za 20 komada - Novi broj komada je vidljiv u GET zahtjevu

Dobiveni izlaz: Uspjeh. Zalihe su uspješno ažurirane. Panel koji je imao 10 komada sada ima 30 komada na zalihi.

Ispit 4: Pokušaj prilagodbe zaliha s negativnim rezultatom (rubni uvjet)

Ulazi:

Podaci: - Panel ID: 1 (trenutno ima 5 komada) - Promjena količine: -10 komada

Koraci ispitivanja: 1. Autentificirati se kao administrator 2. Poslati PATCH zahtjev na `/api/storage/panels/1/adjust-stock` s količinom -10 3. Provjeriti status odgovora i poruku greške 4. Provjeriti da zalihe nisu promijenjene

Očekivani izlaz: - API vraća status `400 Bad Request` ili `422 Unprocessable Entity` - Vraća poruku: "Insufficient stock to adjust stock 1 by -10" - Zalihe ostaju nepromijenjene na 5 komada

Dobiveni izlaz: Uspjeh. Sustav je odbio transakciju koja bi rezultirala negativnim zalihama. Zalihe ostaju na 5 komada. Ova zaštita sprječava nekonzistentnosti u sustavu.

Ispit 5: Prijava korisnika s JWT tokenom

Ulazi:

Podaci: - Korisničko ime: "admin@limbox.hr" - Lozinka: "SecurePassword123"

Koraci ispitivanja: 1. Poslati POST zahtjev na `/auth/login` 2. Provjeriti da je vraćen JWT token 3. Koristiti token za pristup zaštićenom endpointu `/api/calculator/orders` 4. Provjeriti autorizaciju

Očekivani izlaz: - API vraća status `200 OK` - Vraća objekt s JWT tokenom - Token sadrži korisničke tvrdnje (user ID, username) - Token omogućuje pristup zaštićenim resursima

Dobiveni izlaz: Uspjeh. Korisnik je prijavljen, JWT token je generiran. Token sadrži sve potrebne informacije i omogućuje pristup zaštićenim API endpointima. Token vrijedi 7 dana.

Ispit 6: Prijava s neispravnom lozinkom (sigurnosni test)

Ulazi:

Podaci: - Korisničko ime: "admin@limbox.hr" - Lozinka: "WrongPassword"

Koraci ispitivanja: 1. Poslati POST zahtjev na `/auth/login` s neispravnom lozinkom 2. Provjeriti status odgovora 3. Provjeriti da token nije generiran

Očekivani izlaz: - API vraća status `401 Unauthorized` - Ne vraća JWT token - Vraća poruku: "Invalid credentials"

Dobiveni izlaz: Uspjeh. Sustav odbija prijavu s neispravnom lozinkom. BCrypt validacija ispravno uspoređuje hash-ove i sprječava neovlašteni pristup. Sigurnosni mehanizam radi kako treba.

Ispit 7: Dohvat svih narudžbi (autorizacija)

Ulazi:

Bez specifičnih podataka, koristi JWT token iz prijave.

Koraci ispitivanja: 1. Prijaviti se kao korisnik i dobiti JWT token 2. Poslati GET zahtjev na `/api/calculator/orders` s tokenom u `Authorization` headeru 3. Provjeriti status i podatke

Očekivani izlaz: - API vraća status `200 OK` - Vraća listu svih narudžbi korisnika - Svaka narudžba sadrži kompletne informacije (ID, naziv, status, rezultat)

Dobiveni izlaz: Uspjeh. Korisnik uspješno dohvaća svoje narudžbe. Sustav vraća listu od 3 narudžbe s kompletnim podacima o optimizaciji i potrebnim zalihama.

Ispit 8: Pokušaj pristupa bez autentifikacije (sigurnosni test)

Ulazi:

Bez JWT tokena u Authorization headeru.

Koraci ispitivanja: 1. Poslati GET zahtjev na `/api/calculator/orders` **bez** JWT tokena 2. Provjeriti status odgovora

Očekivani izlaz: - API vraća status `401 Unauthorized` - Vraća poruku: "Missing or invalid authentication token" - Ne vraća podatke o narudžbama

Dobiveni izlaz: Uspjeh. Sustav ispravno odbija neautentificirane zahtjeve. JWT middleware provjerava pristup svakom zaštićenom endpointu i osigurava da samo prijavljeni korisnici mogu pristupiti podacima.

Ispit 9: Izračun iskoristivosti materijala (poslovna logika)

Ulazi:

Podaci narudžbe: - Ukupna duljina dijelova: 15000mm (10 × 1500mm) - Iskorišteni stock: 20000mm (4 panela × 5000mm)

Koraci ispitivanja: 1. Kreirati narudžbu s gornjim podacima 2. Provjeriti izračunati yield u odgovoru 3. Provjeriti da sustav prikazuje postotak otpada

Očekivani izlaz: - Yield: 75% (15000 / 20000 × 100) - Otpad: 25% (5000mm od 20000mm) - Sustav preporučuje optimizaciju ili uporabu različitih stock duljina

Dobiveni izlaz: Uspjeh. Sustav je izračunao yield od 75%, što znači 25% otpada. Ova informacija je kritična za poslovno odlučivanje i omogućuje vlasniku tvrtke da vidi gdje može uštedjeti materijal i smanjiti troškove.

Ispit 10: Završavanje narudžbe i ažuriranje zaliha (end-to-end test)

Ulazi:

Podaci: - Order ID: 1 - Oznaka za završavanje: `saveWasteFromLayouts: []` (ne spremaj otpad)

Koraci ispitivanja: 1. Autentificirati se kao korisnik 2. Dohvatiti narudžbu s ID-jem 1 3. Poslati POST zahtjev na `/api/calculator/orders/1/finish` 4. Provjeriti da je status narudžbe promijenjen u "Done" 5. Provjeriti da su zalihe panela smanjene za iskorištene količine

Očekivani izlaz: - API vraća status `200 OK` - Status narudžbe: "Done" - Zalihe panela su smanjene za stvarno iskorištene količine - Sustav bilježi povijest promjena zaliha

Dobiveni izlaz: Uspjeh. Narudžba je završena, status promijenjen u "Done". Zalihe su automatski ažurirane - paneli koji su korišteni u proizvodnji su oduzeti od ukupnih zaliha. Sustav održava sinkronizaciju između narudžbi i stanja skladišta.

Zaključak

Provedeno je sveobuhvatno ispitivanje sustava na razini **komponenti** (10 unit testova) i **sustava** (10 integracijsk ih/E2E testova).

Ispitivanje komponenti pokazalo je da: - Autentifikacija i sigurnost rade ispravno (BCrypt, JWT) - Kalkulacije za optimizaciju rezanja su točne - Upravljanje zalihama ima svu potrebnu validaciju - Rubni slučajevi (dijeljenje s nulom, negativne zalihe) su pokriveni

Ispitivanje sustava pokazalo je da: - REST API endpointi rade ispravno - Autentifikacija i autorizacija štite zaštićene resurse - Poslovna logika (kreiranje narudžbi, ažuriranje zaliha) radi bez grešaka - Integracija između komponenti (Auth $\rightarrow$ Calculator $\rightarrow$ Storage) je funkcionalna - Sustav održava konzistentnost podataka u svim scenarijima

Svi testovi su prošli uspješno, što potvrđuje da sustav ispunjava funkcionalne i nefunkcionalne zahtjeve te je spreman za produkcijsku upotrebu.

Alati korišteni za testiranje

- **Gradle** - Automatizacija buildanja i izvođenja testova
- **JUnit** - Framework za unit testiranje
- **Kotlin Test** - Kotlin biblioteka za testiranje s jasnom sintaksom

- **Ktor Test Host** - Alati za testiranje Ktor aplikacija
- **BCrypt** - Sigurno testiranje hash funkcija za lozinke
- **Postman/Insomnia** - Alati za ručno testiranje REST API-ja (sustav testovi)

Pokretanje testova

Unit testovi:

```
./gradlew test
```

Izvještaj:

```
build/reports/tests/test/index.html
```

Pokretanje aplikacije za sustav testove:

```
./gradlew run
```

Status: BUILD SUCCESSFUL - 10/10 testova prolazi

Korištene tehnologije i alati

Cilj ovog poglavlja je jasno i precizno opisati tehnologije korištene u razvoju aplikacije kako bi se olakšalo buduće održavanje, proširenje funkcionalnosti i suradnja unutar razvojnog tima. U nastavku su opisani programski jezici, radni okviri, baze podataka, razvojni alati, alati za razmještaj i cloud platforme korištene u projektu, uz kratko obrazloženje njihova odabira.

Programski jezici korišteni u razvoju aplikacije su **Kotlin**, **Java** i **Python**. Kotlin (verzija 2.2.20) je korišten za razvoj serverskog dijela aplikacije. Odabran je zbog moderne sintakse, stroge tipizacije, potpune interoperabilnosti s Javom te prethodnog iskustva. Posebno su iskorištene Kotlin Coroutines za asinkrono upravljanje zahtjevima, što optimizira performanse servera. Java (verzija 21 LTS) korištena je kao temeljna platforma (Runtime). Odabrana je verzija 21 zbog dugoročne podrške (LTS) i naprednih značajki poput virtualnih niti, što osigurava stabilnost i visoku propusnost aplikacije. Python (verzija 3.11) korišten je za implementaciju **Ludus kalkulatora**. Odabran je zbog bogate podrške znanstvenim bibliotekama i jednostavnosti integracije s matematičkim solverima.

Radni okviri korišteni u projektu su **Ktor**, **FastAPI** te **PuLP**. Backend dio aplikacije razvijen je korištenjem Ktor frameworka (verzija 3.3.0), laganog i fleksibilnog okvira za izgradnju asinkronih web aplikacija i API-ja u Kotlinu. Ktor omogućuje jednostavnu konfiguraciju servera, dobru integraciju s bazama podataka te visoku razinu prilagodljivosti, što ga čini prikladnim izborom za ovakav tip aplikacije. FastAPI (verzija 0.115) korišten je kao komunikacijski sloj za Python servis. Odabran je zbog iznimne brzine i automatskog generiranja OpenAPI (Swagger) dokumentacije, što je olakšalo integraciju s Ktorom.

Python biblioteka PuLP (verzija 2.8.0) korištena je unutar Ludus kalkulatora za modeliranje i rješavanje optimizacijskih problema rezanja panela.

Za pohranu podataka korištena je relacijska baza podataka **PostgreSQL** (verzija 18). PostgreSQL je odabran zbog svoje pouzdanosti, skalabilnosti i podrške za složenije upite, što je važno za rad s podacima vezanima uz materijale, narudžbe i rezultate kalkulacija.

Razvojna okruženja korištena u razvoju aplikacije su **IntelliJ IDEA Ultimate** i **VSCode**. Za upravljanje izvornim kodom korišten je **Git**, dok je **GitHub** korišten kao centralni repozitorij za suradnju unutar tima, praćenje promjena i verzioniranje projekta.

Za ispitivanje rada aplikacije korišteni su sljedeći alati: **Gradle**, **JUnit**, **Kotlin Test**, **Ktor Test Host**, **BCrypt** i **Postman**. Temeljni okviri za pisanje i izvođenje unit testova su JUnit i Kotlin Test. JUnit je odabran zbog svoje stabilnosti i široke podrške, dok Kotlin Test pruža DSL (Domain Specific Language) prilagođen Kotlinu, što omogućuje pisanje čitljivijih i sažetijih testnih slučajeva. Ktor Test Host korišten je iz razloga što omogućuje testiranje kompletnih API ruta bez potrebe za podizanjem stvarog servera na mreži što znatno ubrzava proces testiranja. Postman je korišten za ručno i automatizirano ispitivanje HTTP zahtjeva. BCrypt je korišten za verifikaciju sigurnosnih aspekata aplikacije, specifično za provjeru ispravnosti hashiranja lozinki korisnika, osiguravajući da se osjetljivi podaci nikada ne spremaju u bazu u čitljivom obliku. U Gradleu su postavljeni specifični zadaci koji prilikom svakog pokretanja automatski izvršavaju cijelu testnu skupinu, čime se osigurava da novi kod ne narušava postojeće funkcionalnosti.

Docker je korišten kako bi cijeli sustav bio kontejniziran te kako bi razvojno okruženje identično odgovaralo produkcijskom.

Aplikacija je hostana na **DigitalOcean** platformi koja je odabrana zbog jednostavne konfiguracije i pouzdane infrastrukture, dok je baza hostana na **Neon** Cloud servisu koji pruža visoku dostupnost te automatske sigurnosne kopije.

Dokumentacija za instalaciju i administraciju (Limbox Backend)

Ovaj dokument pruža detaljne upute za postavljanje, konfiguraciju i upravljanje aplikacijom na `progi` grani.

1. Instalacija

Preduvjeti

Osigurajte da su na sustavu instalirani: - **Docker** (najlakši način za pokretanje svih komponenti). - **Git** source control. - **Java 21** (samo ako želite pokretati projekt direktno bez Dockera).

Preuzimanje koda

```
git clone https://github.com/Ludus-solucije/limbox-backend.git
cd limbox-backend
git checkout progi
```

---

2. Konfiguracija (Environment Varijable)

Aplikacija koristi `.env` datoteku za upravljanje osjetljivim podacima i postavkama okruženja.

Postavljanje datoteke

Kopirajte predložak i popunite ga:

```
cp .env.example .env
```

Detaljan opis varijabli

Varijabla	Opis	Primjer / Napomena
`DB_USER`	Korisničko ime za PostgreSQL bazu podataka.	`postgres`
`DB_PASSWORD`	Lozinka za bazu podataka.	`vaša_lozinka`
`DB_URL`	JDBC putanja do baze. Unutar Dockera koristi se ime servisa `postgres`.	`jdbc:postgresql://postgres:5432/limbox_local`
`CALC_SERVICE_URL`	URL interna adresa Python servisa za izračune.	`http://calculator-service:8000`
`OPTICUTTER_API_TOKEN`	API ključ za vanjski servis Opticutter.	*Dostupno u .env.example*
`OPTICUTTER_API_URL`	Bazni URL za Opticutter API.	`https://sandbox.api.opticutter.com`
`GOOGLE_CLIENT_ID`	ID klijenta za Google OAuth prijavu.	*Dostupno u .env.example*
`GOOGLE_CLIENT_SECRET`	Tajni ključ za Google OAuth.	*Dostupno u .env.example*
`OAUTH_GOOGLE_REDIRECT_PATH`	Putanja na backendu na koju Google vraća korisnika.	`/auth/callback`
`JWT_AUDIENCE`	Ciljana publika (Audience) za JWT token.	`http://localhost:8080`

Varijabla	Opis	Primjer / Napomena
`JWT_ISSUER`	Izdavatelj (Issuer) JWT tokena.	`http://localhost:8080`
`JWT_SECRET`	Tajni ključ za potpisivanje JWT tokena. **Obavezno promijeniti u produkciji!**	`secret-tajno`
`MIGRATIONS_ENABLED`	Omogućava automatsko pokretanje Flyway migracija pri startu.	`true` (za prvi put), `false` inače
`OAUTH_ALLOWED_REDIRECT_ORIGINS`	Dozvoljeni URL-ovi frontenda na koje se korisnik vraća nakon prijave.	`http://localhost:5174`

---

3. Pokretanje putem Docker Compose-a

Docker Compose je preporučeni način jer automatski podiže bazu, Python servis i Kotlin backend.

Pokretanje cijelog sustava

```
docker-compose up --build
```

Što se događa prilikom pokretanja? 1. `postgres`: Podiže bazu na portu `5432` i inicijalizira volume. 2. `calculator-service`: Builda i pokreće Python FastAPI servis. 3. `backend`: Kompajlira Kotlin kod, pokreće Ktor poslužitelj na portu `8080` (ili onom definiranom u `APP_PORT`).

---

4. Administracija i Održavanje

Pristup sustavu

- **Swagger UI (Dokumentacija)**: `http://localhost:8080/docs` – ovdje možete testirati API rute.
- **Baza podataka**: Ako je potrebno pristupiti bazi izvana, koristite `DB_URL` s podacima iz `.env`.

5. Deploy na Digital Ocean

Za postavljanje na Digital Ocean preporučuju se dva pristupa:

Digital Ocean App Platform

1. Povežite svoj GitHub repozitorij s DO računom.
2. Odaberite repozitorij `limbox-backend` i granu `progi`.
3. Digital Ocean će prepoznati `Dockerfile` i automatski pokrenuti build.
4. U postavkama aplikacije na DO panelu dodajte sve varijable iz `.env` datoteke kao **Environment Variables**.
5.

Dodati credentialse za bazu hostanu na neondb

FRONTEND

Preduvjeti

Osigurajte da su na sustavu instalirani: - **Node.js** (verzija >= 18.x, za buildanje) - **Git** source control. - **npm** (verzija >= 9.x, za upravljanje paketima).

Preuzimanje koda

```
git clone https://github.com/Ludus-solucije/limbox-frontend.git
cd limbox-frontend
git checkout progi
```

2. Konfiguracija (Environment Varijable)

Aplikacija koristi `.env` datoteku za upravljanje osjetljivim podacima i postavkama okruženja.

Postavljanje datoteke

Kopirajte predložak i popunite ga:

```
cp .env.example .env
```

Varijabla	Opis	Primjer / Napomena
`VITE_API_BASE_URL`	Adresa backenda.	`http://localhost:8080`

3. Pokretanje aplikacije u lokalnom razvojnom okruženju

```
npm install
npm run dev
```

Server se zatim lokalno pokreće.

4. Deploy na Digital Ocean

Za postavljanje na Digital Ocean preporučuju se dva pristupa:

Digital Ocean App Platform

1. Povežite svoj GitHub repozitorij s DO računom.
2. Odaberite repozitorij `limbox-frontend` i granu `progi`.
3. Digital Ocean će prepoznati `Dockerfile` i automatski pokrenuti build.
4.

U postavkama aplikacije na DO panelu dodajte sve varijable iz `.env` datoteke kao Environment Variables.

Pristup aplikaciji na javnom poslužitelju

Kako pristupiti

Korisnici mogu pristupiti aplikaciji putem URL-a. Backendu se može pristupiti preko URL-a. Za pristup dokumentaciji backenda moze se otici na `/docs` putanju

Izrada projektnog zadatka odvijala se kroz nekoliko faza, od početne analize problema i definiranja zahtjeva, preko dizajna korisničkog sučelja i arhitekture sustava, pa sve do implementacije i testiranja aplikacije. Vrijeme izrade projekta bilo je ograničeno, što je zahtijevalo dobru organizaciju rada unutar projektne grupe te jasno definiranje prioriteta funkcionalnosti koje će biti implementirane u prvoj verziji aplikacije.

Tijekom izrade prepoznato je više tehničkih izazova, od kojih se najznačajniji odnosio na razvoj algoritma za optimizaciju iskorištenja materijala. Poseban izazov predstavljalo je pravilno modeliranje problema kako bi kalkulator mogao dati točne i praktično primjenjive rezultate. Također, izazov je bio dizajn intuitivnog korisničkog sučelja prilagođenog korisnicima koji nemaju tehničko predznanje. Svi ti izazovi uspješno su prevaziđeni. Osim toga veliki izazov je bio prvi put raditi na projektu u grupi i uskladiti komunikaciju među članovima, ali nakon nekog vremena naš tim je postao u potpunosti kompatibilan.

Tijekom rada na projektu stečena su brojna znanja iz područja web razvoja, uključujući rad s frontend i backend tehnologijama, izradu korisničkih sučelja, rad s bazama podataka te implementaciju algoritamskih rješenja u stvarnom poslovnom kontekstu. Posebno je unaprijeđeno razumijevanje timskog rada, planiranja zadataka i komunikacije unutar projektne grupe.

Projekt pruža dobre perspektive za nastavak rada unutar projektne grupe, budući da postoji interes tržišta te mogućnost daljnjeg razvoja i komercijalne primjene aplikacije. Nastavak rada mogao bi uključivati proširenje

funkcionalnosti, dodatan trud oko dizajna, poboljšanje postojećih algoritama te prilagodbu aplikacije specifičnim potrebama novih korisnika i firmi.

U ostvarenoj verziji aplikacije nisu implementirane sljedeće funkcionalnosti: napredna analitika i statistički prikaz potrošnje materijala kroz dulje vremensko razdoblje, automatsko generiranje izvještaja, mobilna verzija aplikacije te podrška za više jezika i napredni sustav korisničkih uloga. Ove funkcionalnosti predstavljaju potencijalne smjerove daljnjeg razvoja projekta.

Rev.	Opis promjene/dodatka	Autori	Datum
0.1	Napravljen Home Page	Roko Šimpraga	20.10.2025.
0.2	sastavljena stranica 1. Opis projektnog zadatka	Roko Šimpraga	22.10.2025.
0.3	sastavljena stranica 2. Analiza zahtjeva	Roko Šimpraga	22.10.2025.
0.4	sastavljena stranica 3. Specifikacija zahtjeva sustava	Roko Šimpraga	22.10.2025.
0.5	Promijenjen UML dijagram obrazaca uporabe	Fran Suknaić	23.10.2025.
0.6	Dodan LICENCE i CODE OF CONDUCT	Leon Mesić	23.10.2025.
0.7	Dodan README	Roko Šimpraga	5.11.2025.
0.8	sastavljena stranica 4. Arhitektura i dizajn sustava do organizacije aplikacije	Roko Šimpraga	8.11.2025.
0.9	Dovršio stranicu 4. Specifikacija zahtjeva sustava	Roko Šimpraga	11.11.2025
1.0	Izmijenjeni sekvencijski dijagrami na 3. poglavlje	Fran Suknaić	13.11.2025
1.1	sastavljena stranica 9. Zaključak i budući rad	Roko Šimpraga	15.12.2025
1.2	sastavljena stranica 5. Arhitektura komponenata i razmještaja	Roko Šimpraga	16.12.2025
1.3	sastavljena stranica 7. Tehnologije za implementaciju aplikacije	Roko Šimpraga	17.12.2025
1.4	sastavljena stranica 6. Ispitivanje programskog rješenja	Roko Šimpraga	16.1.2026
1.5	Izmjene 2. i 3. poglavlja	Roko Šimpraga	22.1.2026
1.6	sastavljena stranica 8. Upute za puštanje u pogon	Marko Dodig	23.1.2026
1.7	Izmjena baze u 4. poglavlju	Marko Dodig	23.1.2026

Dnevnik sastajanja

1. sastanak

- Datum: 1.9.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga
- Tema sastanka: > Prezentirana ideja o stvaranju web aplikacije za kompaniju Limbox

2. sastanak

- Datum: 20.9.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga
- Tema sastanka: > Napravljen discord server Ludus solucije

3. sastanak

- Datum: 5.10.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka: > Određene su uloge na projektu te su oformirani timovi > > FRONTEND: Matej Ruklić(lead), Leon Mesić, Fran Suknaić

  BACKEND: Marko Dodig(lead), Petar Vujica

  DESIGN I DOKUMENTACIJA: Roko Šimpraga

4. sastanak

- Datum: 19.10.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Prvi sastanak nakon što je projekt službeno odobren za predmet Programsko Inženjerstvo, napravljeni su prvi issuei za login koji se radi prema napravljenom dizajnu iz figme.

5. sastanak

- Datum: 24.10.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Sastanak održan kako bi napomenuli važnost komunikacije među članovima, dogovoreno da se svi zadani rokovi moraju poštivati. > Zamoljeni su članovi da pomognu u izradi dokumentacije i frontenda.

6. sastanak

- Datum: 27.10.2025.

- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Sastanak održan kako bi vidjeli kako projekt napreduje te da se izjasne problemi. > ZAKLJUČAK: cjelokupni dizajn je gotov, izrada se mora ubrzati.

8. sastanak

- Datum: 5.11.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Razgovara se o spremnosti baze i implementaciji OAuth.

9. sastanak

- Datum: 11.11.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Prolazi se kroz kod kako bi završili implementaciju OAuth.

10. sastanak

- Datum: 20.12.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Dogovor o rokovima za isporuku featurea.

11. sastanak

- Datum: 30.12.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Rasprava o stanju razvoja novih featurea.

12. sastanak

- Datum: 7.1.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Rasprava o stanju razvoja novih featurea.

13. sastanak

- Datum: 22.1.2025.
- Prisustvovali: Marko Dodig, Matej Ruklić, Roko Šimpraga, Petar Vujica, Leon Mesić, Fran Suknaić
- Tema sastanka:

  Finalno sređivanje aplikacije.

Plan rada

1. Ciklus

2. Ciklus

Tablica aktivnosti

Aktivnost	Roko Šim-praga	Matej Ruklić	Leon Mesić	Marko Dodig	Petar Vujica	Fran Suk-naic
Upravljanje projektom	4	4	1	30	3	
Opis projektnog zadatka	1			2		
Funkcionalni zahtjevi	1			4		
Opis pojedinih obrazaca	1					
Dijagram obrazaca	1		1	1		2
Sekvencijski dijagrami			1	1		1
Opis ostalih zahtjeva	1					
Arhitektura i dizajn sustava	2			10		
Baza podataka	1			4	1	
Dijagram razreda	1			1		
Dijagram stanja	2					
Dijagram aktivnosti	2					
Dijagram komponenti	2					
Korištene tehnologije i alati	2	2		10		
Ispitivanje programskog rješenja	2	5		5		

Aktivnost	Roko Šimpraga	Matej Ruklić	Leon Mesić	Marko Dodig	Petar Vujica	Fran Suknaic
Dijagram razmještaja	2					
Upute za puštanje u pogon	2			10		
Dnevnik sastajanja	1					
Zaključak i budući rad	1			5		
Popis literature				1		
Izrada početne stranice			3			4
Izrada baze podataka				20		
Spajanje s bazom podataka				10		
Izrada aplikacije – backend/frontend		100	75	120	10	75
Izrada prezentacije						

Dijagram pregleda promjena

Ključni izazovi i rješenja

- problemi sa implementacijom Googleovog OAuth2.0
- neujednačen stil pisanja koda
- deployanje aplikacije na javni servis
- extractanje tajnih ključeva u env varijable
- Programsko inženjerstvo, FER ZEMRIS, http://www.fer.hr/predmet/proinz
- Baze podataka, FER ZPR, https://www.fer.unizg.hr/predmet/bazpod
- Razvoj programske potpore za web, FER ZPR/ZARI, https://www.fer.unizg.hr/predmet/web1
- Drawio, https://www.drawio.com
- Astah Community, http://astah.net/editions/uml-new
- Shadcn, https://www.shadcndesign.com
- Figma, https://figma.com
- React, https://react.dev/learn
- ReactBits, https://reactbits.dev

Programsko inženjerstvo ak.god 2025./2026

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

Limbox

Tim: <TG05.1>

Ludus Solucije

Nastavnik: Vlado Sruk